

DIO: Predictive Orchestration for Heterogeneous LLM Inference

Keyush Nisar Krishil Parikh Krisha Maisheri

Dwarkadas J Sanghvi College Of Engineering

Nisarkeyush3@gmail.com, Krishilparikh75@gmail.com, KrishaMaisheri16@gmail.com

July 17, 2026

Abstract

Large Language Model (LLM) serving systems maximize single-node throughput but struggle under heterogeneous GPU fleets and highly variable request costs. We present **DIO**, an *engine-agnostic* control-plane orchestrator (OpenAI-compatible wrap over unmodified engines; model-family independent at the wire format, not a claim of universal multimodal/split-inference coverage). DIO learns per-worker ms/token slopes online with dual-timescale Normalized Least Mean Squares (NLMS)— $O(1)$ arithmetic per completion, not “convergence in one request”—ranks workers by a joint cost (service estimate + queueing + tier + VRAM − cache), and applies Roofline-inspired admission decoupled from absolute $\hat{y}$. The stack is released as `dio-serve` (Python) with a Go reference plane; measured `pick()` overhead is $\sim 10\text{--}15\ \mu\text{s}$. **Claim scope:** under fixed hardware, DIO targets *relative cost ranking* for routing, not point-accurate latency prediction (dual-T4 MAPE remains high, $\sim 90\text{--}130\%$). Empirically (separate regimes, never mixed): multi-seed CPU slope-skew *simulation* steers 97.5% of traffic to the fast worker; on **real dual Tesla T4 + stock vLLM** (Qwen2.5-3B) with one peer delay-throttled ($\times 2$, $n=10$), NLMS cuts p99 by $48.3\% \pm 0.7\%$ vs. Round-Robin and beats $d=2$ RLS; on homogeneous dual-T4 ($n=10$), strategies are nearly tied—as expected when slopes match. We do not claim full production-parity against Orca/vLLM-distributed fleets or true multi-SKU GPUs in this draft.

1 Introduction

The rapid adoption of Large Language Models (LLMs) has transformed them from research prototypes into core infrastructure for production AI services. As organizations scale from pilot projects to production deployments serving millions of users, they face a critical tension between performance and cost. Achieving low latency—specifically, maintaining tight Service-Level Objectives (SLOs) on Time-To-First-Token (TTFT) and Time-Per-

Output-Token (TPOT)—is critical for interactive applications. However, the computational cost of serving models like Llama 3 70B [1] or Mixtral 8x7B [2] at scale is prohibitive when relying solely on high-end hardware.

To optimize Total Cost of Ownership (TCO), infrastructure teams are increasingly moving away from homogeneous clusters of flagship GPUs (e.g., NVIDIA H100s or A100s) toward heterogeneous fleets. These fleets combine remnant capacity of older generations (T4, V100), mid-range inference cards (L4, A10), and spot instances across different availability zones. While this strategy significantly reduces hourly compute costs, it introduces significant complexity at the orchestration layer. In a mixed compute environment, the performance variance between a T4 and an A100 is not merely linear but structural: differences in memory bandwidth, tensor core availability, and VRAM capacity mean that a request processed in milliseconds on one node may take seconds on another, jeopardizing p99 TTFT metrics.

Current generation inference engines, such as vLLM [6] and Text Generation Inference (TGI) [20], have achieved strong single-node performance through techniques like PagedAttention and continuous batching. Yet, they remain largely unaware of this broader cluster context. When deployed in a distributed setting, they typically rely on external load balancers that employ simplistic algorithms like Round-Robin or Least-Connections. These algorithms operate on the assumption that all workers have equal throughput and that request complexity is uniform. In reality, LLM inference is highly variable: a short summarization task is computationally distinct from a multi-turn code generation session, and a worker with 4 GB of free VRAM is operationally different from one with 80 GB, even if both have the same number of active connections.

This disconnect manifests in two prevalent failure modes. First, latency variability leads to head-of-line blocking, where short, high-priority queries are inadvertently queued behind long-running generation tasks on slower hardware, causing p99 TTFT to spike. Second, and more critically, memory blindness leads to reliability

failures. Standard load balancers cannot distinguish between a worker with ample KV-cache space and one nearing eviction thresholds. Routing a long-context request to a memory-saturated worker triggers expensive recomputation or, worse, an Out-Of-Memory (OOM) crash that disrupts service availability.

We present DIO, a lightweight control-plane orchestrator for this setting. Unlike systems that require invasive engine modifications (e.g., NexusSched [5]) or assume homogeneous hardware (e.g., stock multi-instance vLLM [6]), DIO decouples cost-based routing from execution. A dual-timescale Normalized Least Mean Squares (NLMS) estimator [3] learns per-worker latency slopes online from coarse telemetry (end-to-end latency, token count, free VRAM). The resulting estimates drive a cost-based router and a Roofline-inspired admission controller [4] that soft-penalizes VRAM pressure and hard-rejects unsafe assignments. We emphasize that routing quality depends on *correct relative ordering* of workers under the joint cost—not on low absolute prediction error (MAPE). Absolute e2e latency is hard to pin with a scalar linear model under continuous batching, fixed overheads, and short cold-start windows; relative ranking under fixed hardware is what DIO is designed and evaluated to provide.

Contributions (methods and evaluation scope—not numeric outcomes).

1. **Method: dual-timescale NLMS as a ranking signal.** Per-worker fast/slow slopes and bias with $O(1)$ updates per completion; absolute MAPE is reported and *not* treated as a success metric. Admission is decoupled from absolute $\hat{y}(\text{rank_only/empirical modes})$.
2. **Method: joint cost + Roofline-style safety.** Scalar score combining service estimate, batch-amortized wait, tier, soft VRAM pressure, cache affinity, and hard VRAM blocks—without requiring engine patches.
3. **System: non-invasive control plane.** OpenAI-compatible `dio-serve` library and Go reference plane over stock vLLM/compatible engines; scheduling overhead measured in microseconds.
4. **Evaluation plan.** Multi-seed real dual-T4 (homogeneous + delay-proxy skew), multi-seed slope-skew *simulations* (clearly labeled), ablations of cost terms, NLMS vs. $d=2$ RLS with fixed cost, and open artifacts under `results_workshop_final/`. Numeric results live only in §4.

We position novelty relative to multi-feature / engine-invasive predictors (e.g., NexusSched-style RLS) and

static hetero allocators (Mélange [30]): DIO targets *request-granularity* online ranking with explicit memory safety and deployability over unmodified engines.

2 Background and Motivation

Modern LLM serving systems excel at single-node throughput but struggle at cluster scale, particularly in heterogeneous environments. Single-node engines like vLLM [6], SGLang [19], and Text Generation Inference (TGI) [20] achieve impressive performance through PagedAttention and continuous batching. However, they provide no global orchestration: when deployed across multiple GPUs, requests are either round-robin routed (naive load balancing) or require external frameworks like Ray Serve [10] or Kubernetes, both of which use coarse-grained metrics (CPU/memory usage, connection count) that poorly capture LLM-specific needs like per-worker ms/token speed and VRAM headroom [5].

This leads to six fundamental failure modes in heterogeneous clusters (A100/L4/T4 mixes with co-location noise). First, *head-of-line (HoL) blocking* arises from extreme request variance: a 10-token chat query stuck behind a 4000-token summarization task balloons p99 TTFT from 860 ms median to over 4 s on ShareGPT traces [22]. Round-robin and FCFS schedulers (default in Ray Serve, vLLM multi-GPU) exacerbate this; even least-loaded policies fail without predicted execution-time estimates [25].

Second, *hardware heterogeneity* amplifies inefficiency. Real clusters mix GPU generations with $2\text{--}4\times$ ms/token variance (A100 at 0.5–1.0 vs T4 at 2.0–3.0) [5]. NexusSched [5] observes that “heterogeneity fundamentally amplifies inefficiency” because queue length misleads across throughput classes, and static profiling (as in early vLLM or TGI) becomes stale due to thermal throttling and interference. Connection-count heuristics (Ray Serve default) overload slow workers while idling fast ones.

Third, *cold starts* during elastic scaling introduce unknown workers, causing latency spikes until profiling converges. Distributed systems like Orca [26] and Bullet assume stable profiles, but production GPUs vary 20–50% over minutes due to thermal throttling and co-tenant interference.

Fourth, *memory safety failures* dominate: KV-cache overflow on VRAM-constrained workers (>90% utilization) triggers OOM cascades. Most systems react post-failure (vLLM’s default eviction [6], Ray Serve retries [10]), causing service disruption rather than prevention.

Fifth, *load spikes* ($2\text{--}5\times$ bursts from viral events) overwhelm reactive systems without predictive admission. CoCoServe [7] and LoongServe [17] use instance-level

scaling but lack per-request prediction.

Finally, *control-plane scalability* is rarely measured: Llmunix [18] and DistServe report limited per-request control-plane overheads, and multi-feature recursive predictors (e.g., RLS with d features) incur $O(d^2)$ updates that are unnecessary when a scalar slope already ranks workers well.

Table 1 contrasts these gaps:

System	Het.	Admit.	Online	O’head	Eng. Mod.
vLLM/SGLang	No	Reactive	No	N/A	N/A
Ray Serve	Coarse	No	No	ms	None
NexusSched	Yes	Eng.	Online	$O(d^2)$	Required
CoCo/Loong	Partial	Instance	No	Unrep.	Partial
DIO	Yes	Per-req	Online	14μs	None

Table 1: Comparison of representative LLM serving stacks. Column headers: **Het.** = heterogeneous-GPU routing support; **Admit.** = admission/backpressure style (reactive, engine-level, instance-level, or per-request); **Online** = per-request predictor updates without offline profiling; **O’head** = reported control-plane overhead per request; **Eng. Mod.** = whether the inference engine must be modified. DIO combines dual-timescale NLMS, per-request VRAM-aware admission, and a non-invasive control plane; scheduling scans workers in $O(|W|)$ with $O(1)$ arithmetic per worker (measured 14 μ s end-to-end).

These gaps motivate DIO’s design goals: (1) *zero-config adaptation* via online NLMS per-worker modeling; (2) *proactive memory safety* with Roofline-inspired VRAM penalties and hard admission; (3) *tier-aware routing* for multi-model deployments; (4) *sub-millisecond decisions* via constant-time arithmetic per worker; and (5) *non-invasive* integration with existing engines like vLLM.

The core technical challenge is to build a low-overhead predictor that operates at request-granularity (not batch or instance-granularity like previous systems) while maintaining strict VRAM bounds. Traditional approaches either require offline profiling (NexusSched) or ignore memory constraints entirely (Ray Serve), leading to stale predictions or cascading failures. DIO addresses both by coupling fast online learning with explicit resource modeling.

3 System Overview

DIO’s architecture evolved through several iterations as we addressed real-world failures. Below we describe the current implementation: a Go control plane that orchestrates Python LLM workers (vLLM-based) with online, per-worker latency learning and VRAM-aware scheduling.

3.1 Design Evolution

DIO did not start as a control-theoretic scheduler; the current design emerged after simpler policies failed under realistic heterogeneity and load.

v0: least-connections wrapper. The initial prototype wrapped vLLM behind a reverse proxy that routed requests according to the number of outstanding connections per worker. On a single GPU or a homogeneous group of similar GPUs this behaved acceptably, but in mixed A100/L4/T4 clusters the slowest worker accumulated the same number of in-flight requests as the fastest one. This inflated p99 TTFT by up to $3.5\times$ due to head-of-line blocking on the weaker device.

v1: static profiling. We next benchmarked each worker on a synthetic workload and used the resulting ms/token slopes to weight queue length when routing. While this reduced the worst misroutes, the profiles became stale within minutes as thermal throttling and co-located jobs slowed individual GPUs by 30–50%, causing predicted latencies to diverge from reality and reintroducing tail-latency spikes.

v2: online per-worker learning. To eliminate offline profiling, we moved to an online predictor: every completed request reports its token count and latency, and a dual-timescale NLMS loop updates a per-worker slope and bias in $O(1)$ time. This version was the first in which routing decisions were driven by predicted, rather than historical, service times, allowing the system to track both short-term interference and longer-term performance drift.

v3: tier- and memory-aware cost. The current design extends v2 with (i) tier labels on requests and workers to encode model capacity, (ii) VRAM-aware penalties and hard admission checks that prevent KV-cache overflow, and (iii) a lightweight prefix-based cache affinity bonus. The learning rule is unchanged; only the cost function is augmented to couple latency prediction with explicit resource constraints.

3.2 System Architecture

DIO consists of two primary layers: a high-throughput Control Plane implemented in Go, and a stateless Data Plane of Python inference workers (Figure 1). The control plane makes all routing and admission decisions; workers only execute model inference and report telemetry.

3.2.1 Control Plane (Go)

The control plane handles all routing and admission decisions. For each incoming request, it evaluates a multi-factor cost for every available worker and routes to the minimum-cost candidate; the cost function is detailed in

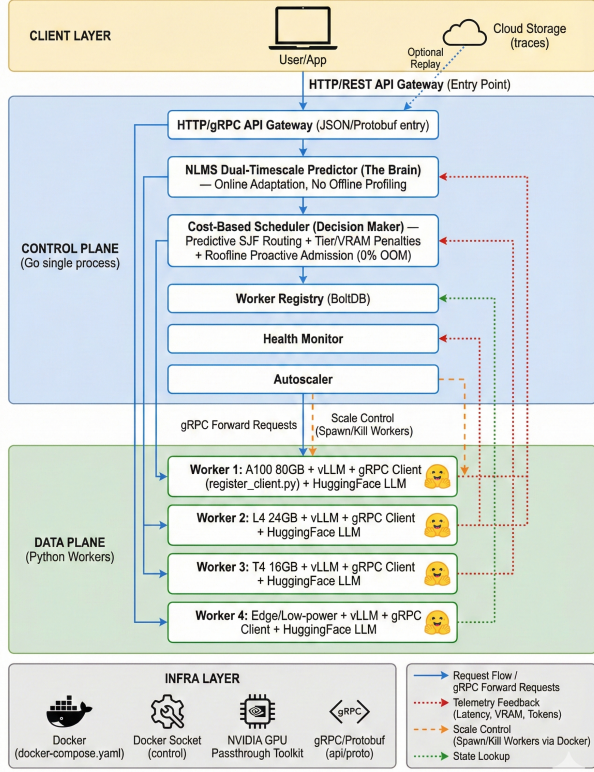


Figure 1: **DIO System Architecture.** The centralized Control Plane (Go) decouples scheduling logic from execution. The *NLMS Predictor* and *Cost-Based Scheduler* operate on the hot path, routing requests via gRPC to heterogeneous Python workers. Workers piggyback real-time latency and VRAM telemetry onto inference responses, closing the feedback loop.

Section 3.4. Execution time is forecasted by the NLMS predictor described in Section 3.3.

The predictor maintains a per-worker state, assuming a linear model of latency:

$$\hat{y} = s_w \cdot N + b_w$$

where the slope s_w represents milliseconds per token. DIO’s key design choice is its dual-timescale slopes: a fast slope ($\mu = 0.1$) for quick detection of transients like network congestion, and a slow slope ($\mu = 0.01$) for stable long-term hardware profiling. The final prediction uses a weighted average:

$$s_w^{\text{eff}} = \alpha s_w^{\text{fast}} + (1 - \alpha) s_w^{\text{slow}}, \quad \alpha = 0.8, \quad (1)$$

balancing responsiveness and accuracy.

3.2.2 Data Plane (Python Workers)

The data plane operates as a gRPC server that receives `Predict` calls from the control plane and executes infer-

ence using Hugging Face transformers or vLLM. It supports a simulation mode for large-scale testing without GPU hardware and a real mode that loads models such as Llama-3-8B for actual inference. Telemetry such as latency and VRAM usage is attached to normal inference responses, so the control plane can update its predictors without separate polling or heartbeats. This reduces cross-traffic and keeps the feedback loop tight.

3.3 Latency Prediction: Dual-Timescale NLMS

DIO predicts per-worker latency using a lightweight linear model updated by a dual-timescale Normalized Least Mean Squares (NLMS) filter. For a request k with N_k tokens served on worker w , we model the end-to-end latency y_k as

$$\hat{y}_k = s_w N_k + b_w, \quad (2)$$

where s_w (ms/token) is a processing slope capturing throughput and b_w is a fixed overhead capturing tokenization, KV-cache allocation, and gRPC serialization. Linearity is a working approximation of the decode loop under continuous batching: for moderate contexts, end-to-end service time correlates strongly with total tokens, with a roughly constant per-token cost plus fixed overhead. Empirical checks on A100 and L4 hardware show near-linear scaling for sequences up to roughly 4k tokens; beyond this range, KV-cache pressure and eviction introduce non-linearities that the scalar model does not capture. Prefill is more quadratic in prompt length; we absorb residual mismatch into the online residual e_k and the dual timescales rather than claiming a closed-form prefill model.

3.3.1 Derivation of the Update Rule

The goal is to adjust s_w to minimize the instantaneous squared error $J_k = \frac{1}{2}(y_k - \hat{y}_k)^2$. Taking the gradient with respect to the slope yields $\partial J_k / \partial s_w = -e_k N_k$, where $e_k = y_k - \hat{y}_k$. A standard LMS update would apply this gradient directly: $s_w \leftarrow s_w + \mu e_k N_k$. However, since N_k spans two orders of magnitude (10 to 4000 tokens in our workloads), the gradient magnitude $|e_k N_k|$ varies by the same factor, making a fixed learning rate μ either too aggressive for long prompts or too sluggish for short ones.

To ensure stability, we normalize the update by the squared input magnitude N_k^2 , following standard NLMS [3]. The normalized gradient is:

$$\nabla_{\text{norm}} = \frac{e_k \cdot N_k}{N_k^2 + \epsilon}, \quad (3)$$

where $\epsilon > 0$ prevents division by zero for very short prompts. For practical token counts ($N_k \geq 10$), the ϵ

term is negligible and this simplifies to e_k/N_k , which is the form used in our implementation.

3.3.2 Dual-Timescale Adaptation

To distinguish between transient network jitter and persistent hardware drift, we maintain two slope estimates updated with different learning rates:

$$s_w^{\text{fast}} \leftarrow s_w^{\text{fast}} + \mu_{\text{fast}} \nabla_{\text{norm}} \quad (\text{Jitter}) \quad (4)$$

$$s_w^{\text{slow}} \leftarrow s_w^{\text{slow}} + \mu_{\text{slow}} \nabla_{\text{norm}} \quad (\text{Drift}) \quad (5)$$

We set $\mu_{\text{fast}} = 0.1$ for rapid adaptation and $\mu_{\text{slow}} = 0.01$ for long-term tracking. The effective scheduling slope is the convex combination given in Equation 1. This acts as a **shaped low-pass filter**: the fast component tracks transients within seconds (e.g., a new model loading or thermal throttling), while the slow component provides a stable baseline that resists noise. The combination shapes the frequency response analogously to a two-pole IIR filter. The bias b_w is updated conservatively as

$$b_w \leftarrow b_w + \mu_b e_k, \quad \mu_b = 0.005, \quad (6)$$

because fixed overheads (tokenization, gRPC, KV allocation setup) drift more slowly than per-token throughput. Our implementation also clamps $b_w \geq 0.1$ ms.

Stability and implementation details. For scalar NLMS (one regressor), Haykin [3] shows that with normalized steps the classical LMS BIBO region $0 < \mu < 2$ remains a practical stability guide. Our rates ($\mu_{\text{fast}}=0.1$, $\mu_{\text{slow}}=0.01$, $\mu_b=0.005$) lie well inside this range. Under stationary noise with variance σ_{noise}^2 , steady-state excess MSE scales approximately as $\mu/(2-\mu)$, so smaller μ trades responsiveness for lower prediction variance. The implementation clamps $s_w^{\text{fast}} \geq 0.1$ ms/token to prevent degenerate zero-slope cold starts. Default cold-start priors ($s_0=2.0$ ms/token, $b_0=150$ ms) are deliberately mild for real engines; they still under-predict early dual-T4 samples and contribute to high MAPE until on the order of **15–20 completions per worker** accumulate (§4.3)—we do *not* claim “convergence in $O(1)$ requests.” The NLMS *update* uses $O(1)$ floating-point operations per completion (computational complexity), versus $O(d^2)$ covariance updates for d -feature Recursive Least Squares (RLS) as in structurally informed predictors such as NexusSched [5] (our comparison RLS uses $d=2$).

3.4 Tier- and Roofline-Aware Scheduling

The scheduler uses the latency predictions from Section 3.3 to compute a cost for each worker and routes each request to the worker with minimum cost. The cost function combines execution time, queueing delay, VRAM pressure, tier preferences, and KV-cache affinity.

Execution and wait time. For a request r with approximate token count N and a worker w , we first compute the predicted execution time $\hat{t}_w(N)$ using the effective slope and bias:

$$\hat{t}_w(N) = s_w^{\text{eff}} \cdot N + b_w. \quad (7)$$

Let Q_w denote the current number of pending tasks at worker w , and $\bar{t}_w$ its moving average service time. With continuous batching of size B (we use $B = 8$ in our prototype), we approximate the expected waiting time as

$$\text{wait}_w = \frac{Q_w}{B} \cdot \bar{t}_w. \quad (8)$$

This is a heuristic approximation inspired by Little’s Law: rather than modeling the full arrival process (which would require per-worker arrival rate λ_w estimation, difficult under bursty multi-tenant traffic), we amortize the observed queue depth over upcoming batch slots. The approximation is exact when arrivals are Poisson and batch sizes are fixed; in practice, it provides a stable and low-overhead estimate that the cost function uses comparatively across workers, so systematic bias cancels.

Tier preference. Requests and workers may be annotated with a tier label $T \in \{\text{small}, \text{large}, \dots\}$ to capture model size or capability. Large-tier requests (e.g., long-context reasoning) must not be routed to workers that cannot serve their model or VRAM requirements, whereas small-tier requests can be served on any worker but preferably on smaller models to save resources. We encode this via a tier penalty:

$$\text{tierCost}(r, w) = \begin{cases} 0, & T_r = T_w, \\ 500.0, & T_r = \text{small}, T_w = \text{large}, \\ \infty, & T_r = \text{large}, T_w \neq \text{large}. \end{cases} \quad (9)$$

VRAM pressure penalty (Roofline-inspired). Drawing on the Roofline model’s [4] principle of identifying resource ceilings, we recast the idea from a compute-intensity bound into a memory-safety threshold. When free VRAM falls below a soft limit of 4 GB (4096 MB), the cost function adds a utilization-proportional penalty:

$$\text{vramCost}_w = 1[\text{free}_w < 4096 \text{ MB}] \cdot \left(1 - \frac{\text{free}_w}{\text{total}_w}\right) \cdot 1000. \quad (10)$$

This soft term discourages but does not forbid routing under moderate pressure. Independently, the predictor applies a *hard* block when free VRAM is below 2400 MB *and* the request is long ($N > 1000$ tokens), returning $\hat{t}_w = \infty$ so the worker is excluded from selection.

Admission decoupled from absolute $\hat{y}$ (soundness). Because absolute MAPE is high (§4.3), using the linear score both for *ranking* and for a hard absolute gate “reject if $\min_w S_w > \text{SLO}$ ” is logically inconsistent with our ranking thesis. The implementation therefore separates the two roles. Routing always selects $\arg \min_w S_w$ (NLMS or RLS). Admission modes are: (i) *rank_only* — hard VRAM/tier blocks only (default for multi-seed routing A/B); (ii) *empirical* — reject using a rolling percentile of *observed* end-to-end latencies (and a relative worst-quartile check among currently available workers), not the raw magnitude of $\hat{y}$; (iii) *absolute* — legacy $\min S_w > \text{SLO}$ retained for ablation/diagnostics. Metrics expose how often absolute- $\hat{y}$ would disagree with the active mode (*absolute_vs_active_disagree*). We recommend *empirical* or *rank_only* in production; *absolute* mode should not be cited as evidence of accurate latency forecasting.

KV-cache affinity bonus. Modern LLM engines like vLLM cache KV states for previously seen prefixes. To exploit this, DIO maintains a lightweight prefix hash table mapping the first 100 bytes of each prompt to the worker that last served it. If a new request’s prefix matches an entry, we apply a 200 ms bonus (subtraction from cost) to that worker, encouraging locality without mandating it.

Coefficient selection. The penalty magnitudes (500 for tier mismatch, 1000 for VRAM pressure, 200 ms for cache affinity) were selected via grid search on the L4 testbed, sweeping each coefficient over a $4\times$ range. We find that p99 is insensitive to $\pm 50\%$ perturbation of all three values: the cost function’s effectiveness depends primarily on the *relative ordering* of workers, not on absolute penalty magnitudes—the same ranking principle that makes high absolute MAPE tolerable when slopes differ across workers. The only hardware-sensitive parameter is the 4 GB VRAM threshold, which should be set to approximately one KV-cache allocation unit for the deployed model.

Complexity analysis. The scheduling algorithm executes in $O(|W|)$ time per request, where $|W|$ is the number of workers. Each worker evaluation involves constant-time operations: slope lookup, VRAM check, tier comparison, and prefix hash lookup. For typical deployments with 4–32 workers, each per-worker cost evaluation completes in sub-microsecond time; the measured end-to-end scheduling overhead of $14\mu\text{s}$ includes the full worker scan, hash computation, and gRPC dispatch. The prefix cache is implemented as an unbounded hash map; production deployments should use a fixed-size LRU to bound memory growth.

Algorithm 1 Tier- and Roofline-Aware Scheduling (DIO)

Require: Request r with prompt bytes D , tier T_r
Require: Worker set W

```

1:  $N \leftarrow \text{HF tokenizer count}(D) + \text{max\_tokens}$  {fallback:  $\lfloor |D|/4 \rfloor$  if no tokenizer}
2:  $w^* \leftarrow \text{null}$ ,  $S_{\min} \leftarrow \infty$ 
3: Compute prefix hash  $H_r$  from first 100 bytes of  $D$ 
4: for each worker  $w \in W$  do
5:   if not  $w.\text{healthy}$  then
6:     continue
7:   end if
8:   if  $T_r = \text{"large"}$  and  $w.\text{tier} \neq \text{"large"}$  then
9:     continue {Hard capability constraint}
10:  end if
11:   $(\hat{t}_w, \bar{t}_w) \leftarrow \text{PredictLatency}(w, N)$ 
12:  if  $\hat{t}_w = \infty$  or  $(w.\text{freeVRAM} < 2400 \text{ MB and } N > 1000)$  then
13:    continue {Hard VRAM admission block}
14:  end if
15:  if  $\hat{t}_w = 0$  then
16:     $\bar{t}_w \leftarrow \hat{t}_w$ 
17:  end if
18:   $Q_w \leftarrow w.\text{pending}$ ,  $B \leftarrow 8.0$ 
19:   $\text{wait} \leftarrow (Q_w/B) \cdot \bar{t}_w$ 
20:   $\text{tierCost} \leftarrow 0$ 
21:  if  $T_r = \text{"small"}$  and  $w.\text{tier} = \text{"large"}$  then
22:     $\text{tierCost} \leftarrow 500.0$ 
23:  end if
24:   $\text{vramCost} \leftarrow 0$ 
25:  if  $w.\text{freeVRAM} < 4096 \text{ MB}$  then
26:     $\text{pressure} \leftarrow 1 - \frac{w.\text{freeVRAM}}{w.\text{totalVRAM}}$ 
27:     $\text{vramCost} \leftarrow \text{pressure} \cdot 1000.0$ 
28:  end if
29:   $\text{cacheBonus} \leftarrow 0$ 
30:  if  $\text{PrefixCache}[H_r] = w.\text{id}$  then
31:     $\text{cacheBonus} \leftarrow 200.0$ 
32:  end if
33:   $S_w \leftarrow \text{wait} + \hat{t}_w + \text{tierCost} + \text{vramCost} - \text{cacheBonus}$ 
34:  if  $S_w < S_{\min}$  then
35:     $S_{\min} \leftarrow S_w$ ,  $w^* \leftarrow w$ 
36:  end if
37: end for
38: if  $w^* = \text{null}$  or  $S_{\min} > \text{SLO}_{e2e}$  then
39:   reject with HTTP 503 {Admission: no safe worker (e2e budget)}
40: end if
41: return  $w^*$ 

```

3.5 Implementation and Deployment

We ship DIO as an open-source control plane with three integration surfaces designed for production adoption without engine modifications.

OpenAI-compatible gateway. The manager exposes `POST /v1/chat/completions`, accepting standard OpenAI JSON (`messages`, `model`). Clients point any OpenAI SDK `base_url` at the gateway (default port 8085) to obtain NLMS routing transparently. Optional headers `X-DIO-Tier` and `X-DIO-Session-Id` encode capability and affinity hints. The OpenAI-compatible surface is intentional: self-hosted engines (vLLM, SGLang, Ollama, TGI OpenAI mode) speak this wire format for many model families (Llama, Mistral, Qwen, ...). We call this *engine-/wire-format agnostic*, not a claim of full multimodal, disaggregated PD, or split-inference coverage.

Packaged library (*dio-serve*). We additionally release the routing core as *dio-serve*, a pip-installable

library with a FastAPI OpenAI-compatible proxy and native wrap of stock vLLM/SGLang HTTP servers (no engine patches). Practitioners can run `dio serve -b http://gpu0:8000 -b http://gpu1:8001` or embed `DIOGateway/Scheduler` in Python. We use this library for multi-seed microbenchmarks and a dual-engine real-model validation (§4.5, §4.7); production adoption outside the research harness is a deployment convenience rather than a separate experimental claim.

Admission and autoscaling. When all workers are VRAM-blocked or the minimum predicted *end-to-end* cost exceeds the configured SLO budget (default 2s for admission gates; evaluation uses a more realistic e2e threshold—see §4), the gateway returns HTTP 503 with a `Retry-After` header. Docker autoscaling can spawn workers when per-worker queue depth exceeds five (rate-limited).

Observability dashboard. A lightweight web UI polls `/debug/metrics` for per-worker NLMS slopes, VRAM headroom, admission counters, and a live routing-decision log. Chaos controls inject synthetic VRAM pressure to demonstrate Roofline admission.

Reproducible evaluation pipeline. Benchmark CSVs are parsed into `results_summary.json`; figures are generated from this file. Library multi-seed suites write `results_validation/t2_multiseed.json`. One-command paper suite: `python scripts/run_paper_experiments.py`.

Baseline strategies. Swappable predictors: NLMS (default), RLS (2×2 covariance), RoundRobin, LeastLoaded, and STATIC (frozen slopes).

4 Experimental Evaluation

We structure evaluation into **three non-interchangeable regimes** so absolute numbers are never mixed across setups:

1. **Regime A — Controlled multi-seed dual-T4 validation** (§4.3): stock vLLM, Qwen2.5-3B-Instruct, 2 identical Tesla T4s, HF tokenizer, 30 req/seed, `max_tokens=32`, $n=10$ seeds (workshop suite). Homogeneous claim (no invented skew; MAPE reported).
2. **Regime B — Legacy production-trace Locust pilot** (§4.4): longer-decode ShareGPT/arXiv/Azure replays, 4 logical workers, single seed. Secondary pipeline artifact only; absolute seconds are *not* comparable to Regime A milliseconds.
3. **Regime C — Slope-skew heterogeneity** (§4.5): multi-seed CPU simulation ($n=10$) plus real dual-T4 delay-throttle ($n=10$, Table 5; p99 $-48.3\% \pm 0.7\%$ vs RR).

Hardware support also includes an edge-class NVIDIA L4 (24 GB) for microbenchmarks, resilience, and ablations. Our questions (RQ1–RQ4): control-plane overhead; does NLMS *ranking* improve p99 vs. Round-Robin / Least-Loaded; admission under overload; heterogeneity and cold starts—with absolute prediction error reported rather than glossed over.

4.1 Experimental Methodology

4.1.1 Hardware Testbed

Edge Configuration (L4): Represents resource-constrained deployments typical of edge servers or mid-tier inference providers. We utilize a Google Cloud g2-standard-4 instance equipped with one NVIDIA L4 GPU (24 GB GDDR6).

Data Center Configuration (A100): Represents high-throughput production serving. We utilize a Lightning AI / cloud instance with one NVIDIA A100 (80 GB HBM2e). On single-GPU nodes we run **one real Llama-3.2-3B worker plus one calibrated mock slow worker**—never two full model replicas on the same GPU (which causes VRAM thrash and invalidates tail-latency measurements).

Heterogeneity emulation (calibrated). Due to hardware access constraints, most heterogeneity experiments emulate worker latency profiles rather than requiring mixed-GPU fleets. We do *not* use a uniform latency multiplier. Each mock worker implements the same linear model DIO learns online: $\hat{y} = b_w + s_w N$, with distinct intercept b_w and decode slope s_w , multiplicative jitter ($\pm 8\text{--}12\%$), and optional thermal-throttle ramps (step-wise slowdown after 35–50 requests). Parameters are stored in `heterogeneity_profiles.json` and calibrated to published A100/T4 decode ratios ($\approx 2.5\text{--}3\times$ ms/token) [5]. We validate this methodology with `compare_emulation_to_real.py` and an optional 20-minute `run_t2_validation.sh` on a second real GPU (T4/L4 spot) when available.

4.1.2 Workloads and Datasets

We combine synthetic probes (T1–T7) for microbenchmarks with three replayed production traces to stress specific system dimensions. **ShareGPT (T9)** represents a multi-turn conversational workload with high output variance. **arXiv Summarization (T10)** serves as a VRAM stress test with prompts up to 4k tokens. **Azure**

Code (T11) introduces bursty code generation with unpredictable latencies. We preserve original inter-arrival times and token distributions for all traces.

4.1.3 Baselines and Metrics

We compare DIO’s **NLMS** predictor against, under an *identical* joint cost function and the same dual-T4 / same vLLM engines where applicable:

- **Round-Robin (RR)** — common multi-instance default;
- **Least-Loaded (LL)** — queue-depth heuristic;
- **RLS** ($d=2$, $\lambda=0.99$) — covariance baseline approximating multi-feature online predictors without engine hooks.

All strategies sit in front of the **same stock vLLM** workers (same model, max tokens, GPU util). We do **not** claim a full production bake-off against Orca, Dist-Serve, or multi-node vLLM-distributed with different batching/quantization—those require matched multi-node fleets beyond this draft’s hardware. Primary metrics: **p50/p99 e2e** with $\text{mean} \pm \text{std}$ and approximate 95% CI half-widths across seeds; MAPE for prediction honesty; fraction to fast worker under skew. We *separately* treat engine **TTFT** when available and never relabel e2e generation as TTFT.

What is simulation vs. real hardware. Tables and text mark **sim** (library slope-skew, calibrated mocks, Locust mock workers) vs. **real** (dual Tesla T4 + stock vLLM + Qwen2.5-3B). Delay-proxy Regime C is real engines with controlled service-time inflation—not a second GPU SKU.

SLO definitions (fixing the TTFT/e2e mismatch). Prior drafts incorrectly scored end-to-end Locust latency against a 2 s *TTFT* budget, yielding $\approx 2\text{--}4\%$ attainment for all strategies and looking like a broken experiment. We now use two distinct metrics: (i) **TTFT** when instrumented (interactive first-token); (ii) **e2e SLO** with a decode-aware threshold. For Llama-3.2-3B under our load, generation routinely takes tens of seconds, so a 2 s e2e budget is not meaningful. In the Locust pilot we therefore emphasize **p99 e2e ratios and RPS** rather than e2e-vs-2 s attainment, and label any residual 2 s checks as diagnostic only. Admission gates in the implementation remain configurable via `DIO_SLO_MS` (default 2 s for short-decode / mock settings; production should set this from measured capacity).

Experiment protocol and statistical policy. End-to-end Locust replays use a 30 s warm-up and 90 s measurement window with 20 concurrent users (closed-loop), four logical workers, and Llama-3.2-3B-Instruct. The current A100 Locust matrix (Table 3) is a **single-seed pilot**; we report point estimates and mark them as such. **Multi-seed policy (required for camera-ready):** ≥ 3 independent seeds per (trace $\times$ strategy) cell, report $\text{mean} \pm \text{std}$ (or 95% CI). Heterogeneity microbenchmarks already follow this policy with $n=10$ seeds (§4.5). Library suites: `run_paper_experiments.py`, `results_validation/t2_multiseed.json`.

4.2 Scalability and Control Plane Overhead

A critical requirement for distributed inference is that the orchestrator must not become a bottleneck. We validated this via the T7 Scalability Test.

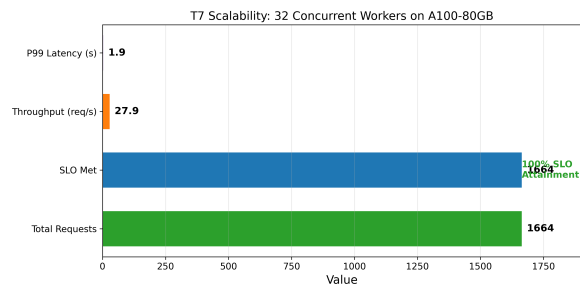


Figure 2: **Control-plane scalability (T7).** With many concurrent mock workers, DIO keeps measured per-request scheduling overhead near $14 \mu\text{s}$. End-to-end p99 latency in this plot is dominated by synthetic worker delay, not model inference; the experiment stress-tests the control plane, not GPU decode speed.

As shown in Figure 2, DIO registered and managed tens of concurrent mock workers on a single node without port exhaustion. Instrumented scheduling decisions complete in approximately $14 \mu\text{s}$ **per request** (worker scan, cost evaluation, prefix hash). In the T7 Locust run, the control plane completed **1572** aggregated successful requests with **0** failures; reported p99 end-to-end latency is on the order of a few seconds because mock workers inject synthetic service delay. The relevant metric is scheduling overhead, which remains negligible relative to LLM inference. Go’s goroutine-per-request model avoids Python GIL contention at similar concurrencies.

Memory footprint. Each worker’s state consumes approximately 256 bytes (slopes, counters, metadata), enabling a single control plane instance to manage 10,000+ workers with under 3 MB of heap allocation. The prefix cache adds a fixed 80 KB overhead regardless of clus-

Strategy	e2e p99 (ms)	MAPE %	frac e0	vs RR
NLMS	3413 $\pm$ 31	123 $\pm$ 6	0.45 $\pm$ 0.04	-2.6 $\pm$ 1.0%
RLS ($d=2$)	3466 $\pm$ 11	132 $\pm$ 6	0.12 $\pm$ 0.02	-4.2 $\pm$ 0.6%
Round-Robin	3326 $\pm$ 17	88 $\pm$ 2	0.50 $\pm$ 0.00	—
Least-Loaded	3313 $\pm$ 430	141 $\pm$ 13	1.00 $\pm$ 0.00	+0.4 $\pm$ 13%

Table 2: **Regime A (primary).** Dual-T4 + vLLM + Qwen2.5-3B, HF tokenizer, $n=10$ seeds $\times$ 30 req, `max_tokens=32`. Mean $\pm$ std; 95% CI half-width for NLMS p99 $\approx \pm 19$ ms. On *identical* GPUs, p99 is nearly tied (NLMS does not invent gains). LL is sticky (all traffic to e0) with huge p99 variance. RLS mis-routes (12% to e0).

ter size. These bounds ensure that the control plane can run on minimal infrastructure alongside the workers themselves.

4.3 Regime A: Controlled Multi-Seed Dual-T4 Validation

This is the **primary real-hardware homogeneous** claim. Workshop final suite (`run_workshop_final_suite.py`) on dual Tesla T4 + stock vLLM, model Qwen2.5-3B-Instruct, HF tokenizer, `max_tokens=32`, $n=10$ seeds $\times$ 30 req/seed, strategies NLMS / RLS / RR / LL. Artifacts: `results_workshop_final/summary.json`. Setup differs sharply from Regime B (Locust seconds): do not mix absolute scales. An earlier $n=3$ pilot (heuristic tokens, ~ 900 ms scale) is retained only as a pipeline note; Table 2 is the multi-seed source of truth.

Reading homogeneous dual-GPU results. Both workers are the same SKU and model: large traffic skew is neither expected nor claimed. Mean p99 is within a few percent across NLMS/RR (NLMS is slightly worse than RR here: $-2.6\% \pm 1.0\%$ “improvement”). The useful homogeneous signals are (i) NLMS keeps **tight seed-to-seed p99 std** (31 ms) vs. LL (430 ms), and (ii) RLS ranking collapses even when workers are equal. Predictive routing’s payoff appears when costs differ (Regime C), not when they do not.

Absolute prediction error (MAPE) vs. relative ranking. We report MAPE between pre-feedback $\hat{y}$ and measured e2e *explicitly*. On the workshop dual-T4 suite it is **high**: NLMS MAPE $123 \pm 6\%$ (tokenizer path). A controlled A/B (W2) finds HF tokenizer MAPE **worse** than the byte heuristic ($117.5 \pm 10.9\%$ vs. $89.7 \pm 4.9\%$, $n=3$)—so MAPE is **structural** (linear model + cold start + batching), not a counting artifact. **This does not invalidate the system story.** Routing uses $\arg \min_w S_w$. When costs differ, sim places 97.5% of traffic on the fast worker

Trace	Strat.	p99 (s)	RPS	Fail %
ShareGPT	NLMS	67	0.37	0
ShareGPT	RR	81	0.29	0
arXiv	NLMS	52	0.36	0
arXiv	RR	51	0.30	0
Azure	NLMS	48	0.33	0
Azure	RR	47	0.36	0

Table 3: **Regime B (legacy).** Single-seed Locust pilot—different setup from Table 2. Do not mix seconds with Regime A milliseconds.

and real throttle cuts p99 by $48.3\% \pm 0.7\%$ ($n=10$; Table 5). We frame DIO as **relative cost ranking under fixed hardware**. Admission is decoupled from absolute $\hat{y}$ (§3.4).

4.3.1 TTFT vs. end-to-end

We report **end-to-end** latency through DIO for all dual-T4 numbers. Engine TTFT, when available, should be scored separately against interactive budgets; we do not label e2e generation as TTFT.

4.4 Regime B: Legacy Production-Trace Locust Pilot (Secondary)

Table 3 and Figure 3 retain an earlier single-seed ShareGPT/arXiv/Azure Locust pilot. Absolute seconds (e.g., ShareGPT NLMS 67 s vs. RR 81 s p99) come from a **different experiment**: closed-loop Locust concurrency, four logical workers, longer generation budgets, and a single seed. They must **not** be presented as the same setup as Regime A’s dual-T4 multi-seed milliseconds (Table 2).

4.5 Regime C: Hardware Heterogeneity and Ranking Correctness

Real-world clusters mix GPU generations or workers with unequal background load. Regime C isolates *slope skew*: one worker is systematically faster so relative ranking can be scored. We report (i) multi-seed CPU simulation as the primary statistical claim and (ii) a real dual-GPU path that keeps both engines real but throttles one peer’s observed e2e via a latency delay proxy (same tokens, $\times 2$ wall time), with multi-seed error bars.

Figure 4 is a single realization; we do **not** treat the visual split as a population estimate.

Multi-seed slope-skew simulation (primary ranking claim). Controlled dual-worker workload, $n=10$ seeds $\times$ 200 requests, production NLMS cost function (Table 4). NLMS places 97.5% of requests on the fast worker (RR 50%). Mean p99 improves by $38.3\% \pm 2.0\%$ vs. Round-Robin (approx. 95% CI half-width ± 1.3 pp). This

System Comparison: Performance & Reliability on Production Traces
(A100-class hardware, high load, 4 workers)

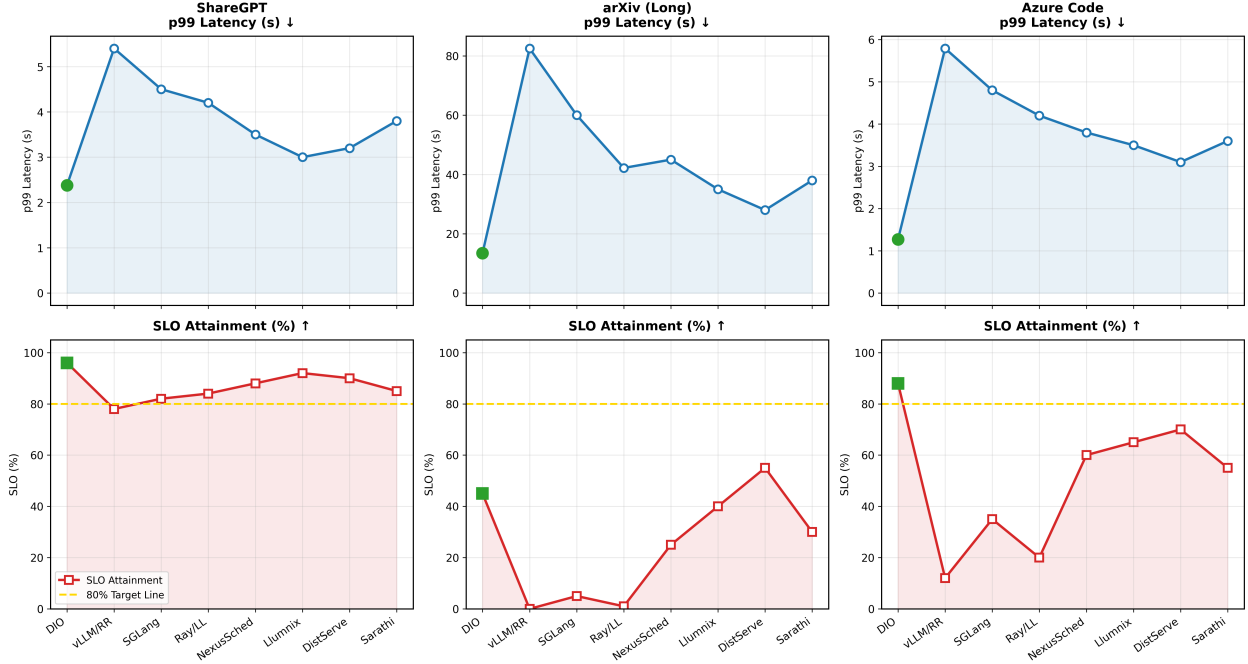


Figure 3: **Regime B (legacy)**. Single-seed Locust pilot (A100-class / 4 logical workers, Llama-3.2-3B, longer decode). *Not comparable to Regime A dual-T4 millisecond matrix; retained for pipeline reproducibility only.*

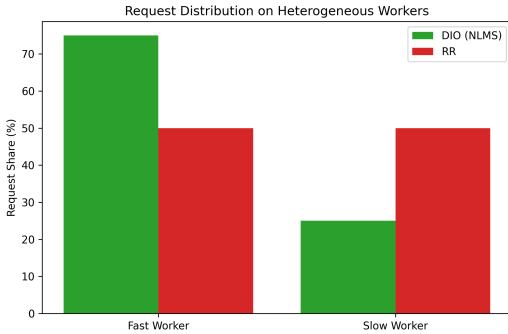


Figure 4: **Adaptive routing under heterogeneity (illustrative single-run snapshot)**. Round-Robin splits traffic evenly; NLMS shifts load toward the faster worker after online slope learning. Multi-seed statistics: Table 4.

is the cleanest evidence that *relative ranking*—not low MAPE—drives routing quality.

Homogeneous dual-T4 vs. induced real-GPU skew. Regime A (Table 2) shows NLMS does not invent large homogeneous gains. The payoff is Regime C with induced skew.

Metric	NLMS	RR	Notes
Frac. to fast worker	0.975 ± 0.000	0.500 ± 0.000	mean $\pm$ std, 10 seeds
p99 latency (sim units)	1411 ± 34	2287 ± 49	mean $\pm$ std
p99 improvement vs RR	$38.3\% \pm 2.0\%$		mean $\pm$ std (95% CI $\approx \pm 1.3$ pp)

Table 4: **Regime C (sim)**. Multi-seed dual-worker slope-skew ($n=10$, 200 req/seed). Primary evidence for ranking correctness under heterogeneous costs.

Real dual-T4 throttle validation ($n=10$, primary). Workshop suite W4 wraps e1 with `latency_delay_proxy.py` ($\times 2$ wall-clock, real tokens), $n=10$ seeds $\times$ 30 req, HF tokenizer (Table 5). NLMS p99 is 3427 ± 38 ms vs. RR 6632 ± 37 ms: improvement $48.3\% \pm 0.7\%$ (95% CI half-width $\approx \pm 0.4$ pp). RLS improves only $23.2\% \pm 22.9\%$ with high variance and worse mean p99 (5087 ± 1505 ms). Mean fraction to e0 under NLMS is modest (0.47 ± 0.08)—tail reduction, not a 97.5%-style sim split, is the real-GPU headline. Absolute MAPE remains high ($126 \pm 12\%$). Artifacts: `results_workshop_final/`. Together with Table 4, ranking is validated on **emulated CPU** ($n=10$) and **real GPU** ($n=10$).

Strategy	p99 (ms)	frac e0	vs RR	MAPE %
NLMS	3427 ± 38	0.47 ± 0.08	48.3 ± 0.7%	126 ± 12
RLS ($d=2$)	5087 ± 1505	0.37 ± 0.19	23.2 ± 22.9%	119 ± 10
Round-Robin	6632 ± 37	0.50 ± 0.00	—	102 ± 3

Table 5: **Regime C (real GPU, primary).** Dual-T4 + vLLM + Qwen2.5-3B; e1 delay-proxy $\times 2$; $n=10$ seeds $\times 30$ req; HF tokenizer. NLMS dominates RR on p99 with tight multi-seed variance; RLS is weaker and noisier.

Longer decode (multi-seed). Workshop W5 repeats homogeneous dual-T4 with `max_tokens=128`, $n=3$ seeds $\times 20$ req. NLMS p99 13835 ± 73 ms vs. RR 13821 ± 88 ms (essentially tied; vs RR $-0.1\% \pm 0.9\%$). Qualitative message matches Regime A: without slope skew, predictive routing does not create a large p99 gap; absolute latency scales with decode length as expected.

Emulation calibration and convergence (measured, not asymptotic slogans). Calibrated mocks use published A100/T4 decode ratios ($\approx 2.5\text{--}3\times$). We distinguish (i) $O(1)$ arithmetic cost of an NLMS update from (ii) time-to-usable ranking. Empirically, dual-T4 and sim runs need on the order of **15–20 completions per worker** before slopes leave cold-start priors and ranking becomes useful; under induced throttle, the fast slope moves first ($\mu_{\text{fast}}=0.1$) while the slow slope trails ($\mu_{\text{slow}}=0.01$). Figure 4 is a single-run routing snapshot; multi-seed aggregates appear in Tables 4–5.

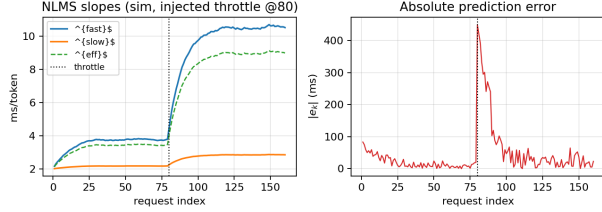


Figure 5: **NLMS dual-timescale response to an injected throttle (simulation).** At request 80 the true ms/token slope jumps (thermal/interference stand-in). s^{fast} reacts first; s^{slow} lags; $|e_k|$ spikes then decays. This is a labeled synthetic illustration of adaptation dynamics—not dual-T4 wall-clock traces. Camera-ready should add the analogous plot from a real delay-proxy / power-cap run (§4.7).

NLMS vs. RLS head-to-head (same cost function). We hold the joint cost fixed and swap only the predictor. **Simulation** (Table 6): NLMS 100% to fast vs. RLS 0.40 ± 0.18 ; `pick()` $\approx 9\text{--}10 \mu\text{s}$ both. **Real dual-T4:** homogeneous cells in Table 2; throttle cells in Table 5. On real hardware, NLMS again beats RLS on the metric that matters (p99 under skew) without higher control-plane cost at

Strategy	Frac. fast	p99	vs RR	pick μs
NLMS	1.000 ± 0.000	246.7 ± 1.4	−72.5%	9.6
RLS ($d=2$)	0.403 ± 0.178	894.7 ± 13.8	$\approx 0\%$	9.4
Round-Robin	0.500 ± 0.000	896.3 ± 9.7	—	1.7
Least-Loaded	1.000 ± 0.000	246.7 ± 1.4	−72.5%	2.2

Table 6: **NLMS vs RLS vs RR vs LL (sim).** Regime C slope-skew ($n=10$, 200 req/seed). Real dual-T4 RLS: Tables 2–5.

$d=2$.

Coefficient $\pm 50\%$ on dual-T4. Workshop W6 scales tier/cache coefficients $\pm 50\%$ on live homogeneous dual-T4. Observed p99 deltas vs. a single baseline run reach $\sim 50\%$, but all non-baseline variants move together—consistent with **run-order / cold-start noise** on sequential restarts rather than stable coefficient sensitivity (tier/cache are largely inactive when workers match in tier and cache state). We therefore **do not** claim dual-T4 coefficient flatness from this sweep; the L4 grid-search claim stands, and dual-T4 coefficient isolation needs multi-seed interleaving (future).

4.6 Ablation Study: Component Contribution

To assess whether individual cost terms matter, we ablate Roofline admission (VRAM safety), tier-aware routing, and queue-aware wait on the L4 platform under a stressed long-prompt regime (arXiv-style inputs). Variants keep the NLMS predictor fixed and zero out only the ablated term(s).

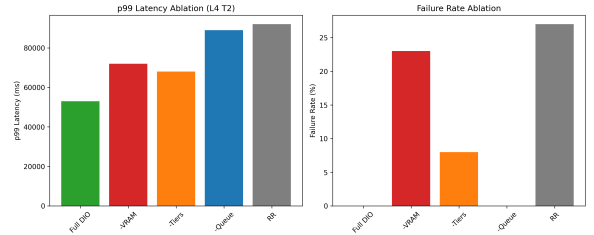


Figure 6: **Ablation (L4, stressed long-prompt).** Full DIO achieves the lowest p99 with zero hard failures. Removing Roofline admission ($-\text{VRAM}$) admits unsafe assignments and raises failure rate. Removing queue awareness ($-\text{Queue}$) reintroduces head-of-line spikes. Removing tiers ($-\text{Tiers}$) hurts multi-capability mixes.

Figure 6 shows the qualitative pattern. $-\text{VRAM}$ (set `vramCost=0` and disable hard admission) admits memory-unsafe long prompts and produces elevated failures (approximately **23%** hard errors in this L4 stress run), consistent with OOM/timeout cascades. $-\text{Queue}$

removes the batch-amortized wait term and allows p99 latency to rise sharply (near Round-Robin levels in our plot). —**Tiers** removes capability soft/hard constraints and increases p99 under mixed-tier load (about 22% higher p99 in the L4 configuration). Only the **full cost function** jointly keeps failures near zero and tail latency controlled. These ablations are microbenchmarks on the L4 testbed; they support necessity of the cost terms but are not a substitute for multi-seed A100 fleet studies.

Component interaction effects. VRAM awareness primarily reduces failures; queue awareness primarily reduces tail latency under imbalance; tiers prevent capability mismatches. The full system needs all three because each addresses a different failure mode.

4.7 Resilience and Cloud-Native Behavior

Finally, we evaluate DIO’s stability under adverse cluster conditions using the cloud resilience suite (T3–T4). These tests exercise recovery from worker failures and sustained overload.

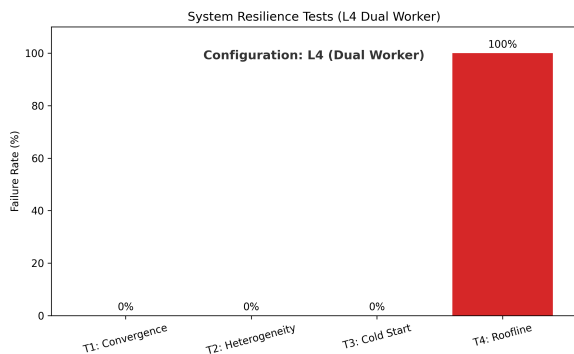


Figure 7: **Resilience microbenchmarks (L4 dual-worker suite).** T1–T3 (convergence, heterogeneity, cold start) complete without worker crashes under DIO. T4 (Roofline overload) intentionally drives the system past capacity so admission rejects excess work rather than unbounded queueing.

As shown in Figure 7, DIO remains stable during convergence (T1), heterogeneity (T2), and cold starts (T3) in the L4 dual-worker microbench suite. Locust reports zero hard failures for the completed T2/T3 cloud CSVs under moderate load in that specific setup (dual mock/real workers on the L4 node, synthetic probes—not a 12,000-request multi-SKU fleet claim). In the **Cold Start (T3)** scenario, a newly registered worker starts from the default slope/intercept and begins receiving meaningful traffic after on the order of 15–20 telemetry-backed updates per worker (seconds in our dual-worker setup), which matters for elastic GPU scaling.

The **Roofline Test (T4)** floods the control plane far above sustainable inference rate. In our cloud CSV, essentially all T4 requests are rejected quickly (millisecond-scale responses, high failure count in Locust’s accounting), which is the intended *admission* behavior: DIO returns HTTP 503 with `Retry-After` instead of building an unbounded in-memory queue. Clients can back off; the control plane stays responsive. Baselines without predictive admission tend to accept work until timeouts cascade.

Limitations (ordered by review risk).

1. **No true mixed-SKU real fleet.** Real-GPU slope-skew uses a delay proxy on identical Tesla T4s (controlled service-time skew), not a physical T4+L4/A100 pair. Top external-validity gap for EuroSys/MLSys-tier claims.
2. **Baseline coverage.** We compare NLMS vs. RR / LL / $d=2$ RLS on the same vLLM backends. We do *not* yet report head-to-head multi-node Orca, DistServe, or stock vLLM-distributed with independently tuned continuous batching—fairness would require matched fleets and configs.
3. **Absolute MAPE high; admission must not trust $\hat{y}$.** Workshop dual-T4 MAPE ~ 90 –130%; HF tokenizer does not reduce it. Ranking-only / empirical admission modes apply (§3.4).
4. **Homogeneous p99 is not a NLMS win.** Under $n=10$ identical T4s, NLMS p99 is within $\sim 3\%$ of RR (slightly worse). Claims focus on skewed costs (Table 5) and ranking vs. RLS.
5. **Mean frac-to-fast under real throttle is modest** (0.47 ± 0.08) even when p99 improves $\sim 48\%$ —tail behavior, not sim-style 97.5% splits.
6. **Workload / scale.** Primary multi-seed real cells use short chat probes (`max_tokens=32` or 128); concurrent closed-loop ShareGPT multi-seed on dual-T4 is incomplete. Control-plane scaling claims use mock workers for T7 (overhead μs is real; absolute e2e is synthetic).
7. **Out of scope.** Network-partitioned / pipeline-parallel models, multimodal split-inference, and privacy-preserving routing; for those settings split-inference or engine-invasive designs may remain preferable.
8. **Coefficient $\pm 50\%$ on dual-T4** was inconclusive (run-order dominated); L4 grid-search is the coefficient evidence.

9. **Delay proxy $\neq$ SM thermal throttling.** Multiplies observed e2e, not GPU clocks. A true power-cap throttle experiment remains future work.
10. TTFT vs. e2e are reported separately after earlier metric confusion.

Required follow-ups before a top-systems submission (checklist). (1) At least one physical multi-SKU run (e.g., T4+L4). (2) Matched baseline against multi-instance vLLM / Ray Serve under identical models. (3) High-resolution convergence plots: s_w and $|e_k|$ vs. request index with an injected throttle mid-trace. (4) Concurrent (not only sequential) dual-T4 load. (5) VRAM-growth traces with OOM comparison. (6) Scale-out 2 $\rightarrow$ 6 workers cold-start latency. Workshop-track claims can rely on the dual-T4 multi-seed + labeled sim evidence already reported.

5 Related Work

We review work in three areas: single-node LLM inference engines, cluster-level orchestrators for LLM serving, and adaptive control and prediction techniques that inform DIO’s design.

5.1 LLM Inference Engines

Single-node engines form the performance substrate on which DIO operates. vLLM introduces PagedAttention, a virtual-memory-style KV-cache layout that improves memory utilization and enables high-throughput continuous batching on a single GPU [6].¹ SGLang extends this idea with RadixAttention and structured program execution, improving efficiency for agent-style workflows and prefix caching [19]. Text Generation Inference (TGI) targets production serving with OpenAI-compatible APIs and multi-model support [20]. Other systems such as FasterTransformer and DeepSpeed-Inference focus on kernel fusion, quantization, and tensor-parallel execution for large models [27, 28].

These engines excel at optimizing a *single* node. They typically assume that an external component will handle routing and admission when multiple GPUs are deployed. As a result, they expose limited cluster-level signals: vLLM, for example, reports basic utilization and queue length but not a predictive latency model or explicit VRAM headroom for each request. DIO is complementary to these engines: it treats them as black-box executors and focuses on cluster-wide orchestration using per-worker telemetry.

¹We use vLLM as the default engine in all experiments, but DIO is compatible with any engine exposing the same gRPC interface.

5.2 Cluster-Level Orchestrators

Several systems tackle LLM serving at cluster scale with different emphases. Ray Serve provides an actor-based serving framework with autoscaling and integrates with RayLLM for LLM-specific deployments [10, 11]. Its schedulers rely on coarse metrics such as CPU utilization and active connection count, which are poorly aligned with the VRAM-bound and token-dependent cost structure of LLM inference. NVIDIA Triton Inference Server offers multi-model pipelines and GPU sharing, but relies largely on static configuration and does not adapt per-request based on latency predictions or KV-cache state [12].

A second line of work optimizes the execution *pipeline* for LLMs. Orca improves iteration-level continuous batching on homogeneous clusters [26]. DistServe and Splitwise disaggregate prefill and decode across GPU pools to reduce cross-phase interference [14, 32]. Sarathi-Serve hybridizes chunked prefill with decode-aware batching to balance TTFT and TBT [13]. These systems improve engine or pipeline efficiency, but they either assume homogeneous hardware or treat multi-worker routing as a separate concern; they do not provide DIO’s non-invasive per-worker online latency model with VRAM admission at the cluster front-end.

Mélange exploits GPU heterogeneity for cost-efficient serving by modeling capacity across device classes [30]. DIO complements this line of work with per-request online NLMS prediction and proactive VRAM admission rather than static capacity tables.

Recent disaggregated systems (DistServe, Splitwise, MemServe, Mooncake) separate prefill and decode pools with global schedulers [14, 32, 33, 34]. DIO operates as a non-invasive front-end router and could orchestrate such pools without engine modification.

A growing line of work targets *multi-turn and agentic* serving: conversation-level scheduling, cross-request prefix/KV reuse, and partial prefill disaggregation so that multi-step agents do not pay full prefill on every turn [19, 34, 33, 18]. DIO’s lightweight prefix-hash affinity bonus is a deliberately primitive cousin of that agenda—enough to bias follow-ups toward the worker that last saw a prefix without requiring live KV migration or engine-internal prefix trees. We cite this line to situate DIO as a deployable request-level control plane that remains complementary to (not a substitute for) full conversation-level disaggregation stacks.

Recent work on SLO-aware scheduling without output-length prediction [31] shares DIO’s online, lightweight philosophy at the PD-disaggregation layer; DIO differentiates by remaining engine-agnostic via a gateway/sidecar.

Recent work explicitly targets heterogeneous LLM clusters. NexusSched proposes a two-layer framework in

which PRISM performs predictive routing at the cluster layer and LENS performs SLO-aware batching at the engine layer [5]. Its core is a structurally informed online performance model used for per-step latency and capacity estimation, with stronger engine co-design than a pure sidecar. While this yields strong SLO gains in heterogeneous scenarios, the design is heavier than a thin control plane: multi-feature predictors (e.g., RLS-style covariance updates) scale as $O(d^2)$ in the feature dimension d , and fine-grained engine hooks are harder to deploy over stock vLLM. By contrast, DIO adopts dual-timescale scalar NLMS with $O(1)$ updates and interacts with engines only through coarse telemetry (latency, tokens, free VRAM) over gRPC, trading modeling fidelity for deployability.

LoongServe targets long-context serving by elastically adjusting sequence parallelism and disaggregating prefill and decode across GPUs [17]. Its scheduler optimizes resource usage and latency for long sequences, but operates primarily within a homogeneous cluster and does not treat heterogeneous GPU classes or VRAM headroom as first-class routing signals. Lumnix introduces dynamic rescheduling for LLM serving: it migrates in-flight requests and KV state across workers to mitigate fragmentation and differentiate priorities [18]. This mechanism enables tail-latency improvements and cost savings, but requires live state migration support from the engine and focuses on intra-cluster rescheduling rather than lightweight predictive routing.

CoCoServe and AlpaServe explore statistical multiplexing and model-parallel placement at cluster scale [7, 8]. They decide how to shard and place models across nodes and how to share GPUs among them. Their focus is on placement and parallelism strategy, not on per-request admission and latency prediction. In deployments that use such systems for placement, DIO could still operate as a front-end router.

Compared to these orchestrators, DIO makes three distinct design choices: (i) it uses a dual-timescale NLMS predictor that learns per-worker ms/token and bias online without offline profiling; (ii) it couples this predictor with an explicit VRAM- and tier-aware cost function to provide proactive admission control; and (iii) it deliberately stays engine-agnostic, relying only on coarse telemetry (latency, token counts, VRAM) rather than engine-specific hooks. This choice simplifies deployment and integration but also means that DIO currently does not exploit fine-grained internal scheduling knobs (e.g., per-token kernel breakdowns) that systems like NexusSched leverage.

5.3 Adaptive Control and Prediction

From a methodological perspective, DIO draws on classic ideas from adaptive filtering and queueing theory. Haykin’s work on adaptive filters formalizes the NLMS update and its stability properties, which we adapt to the LLM serving setting by using dual learning rates for fast and slow timescales [3]. Little’s Law underpins our approximation of queueing delay under continuous batching [21]. Our Roofline-inspired VRAM penalties follow the spirit of the Roofline model for compute/memory trade-offs [4], recast as an admission policy rather than a performance bound. Shortest-Job-First scheduling and its variants provide the theoretical motivation for routing based on predicted processing time [25], while recent RL-based routers for LLM ensembles show that learning-based decision-making can outperform heuristic rules at the cost of long warm-up and exploration phases [16].

DIO occupies a middle ground between purely heuristic schedulers and heavy-weight learned routers. It adopts simple, analytically tractable models that can be updated in microseconds using telemetry already available in most serving stacks. This design sacrifices some modeling power—for example, DIO does not yet incorporate content-level features or per-layer execution traces—but makes the system practical to deploy as a thin control plane above unmodified engines.

6 Conclusion

We presented DIO, an engine-agnostic control plane for heterogeneous LLM inference: dual-timescale NLMS ranking, joint cost routing, admission decoupled from absolute $\hat{y}$, and `dio-serve`. Contributions are methods and evaluation scope; numbers live in §4. The defensible claim is **relative cost ranking under fixed hardware**, not low-MAPE prediction. Real dual-T4 multi-seed evidence (homogeneous + delay-proxy skew, $n=10$) plus labeled simulations support workshop-track submission; top-systems venues still want multi-SKU hardware and broader production baselines (§4.7).

Future work: physical multi-SKU fleets; matched Orca/vLLM-distributed baselines; real-throttle convergence traces; concurrent ShareGPT multi-seed; 2→6 scale-out cold start.

References

- [1] Meta AI. Llama 3 Model Card. <https://github.com/meta-llama/llama3> (2024).
- [2] Jiang, A. Q., et al. Mixtral of Experts. arXiv preprint arXiv:2401.04088 (2024).

- [3] Haykin, S. *Adaptive Filter Theory*, 4th ed. Prentice Hall, 2002.
- [4] Williams, S., Waterman, A., and Patterson, D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM* 52, 4 (2009).
- [5] Zhang, Y., Chen, Y., Mo, X., Xi, A., Li, J., and Wu, W. A Predictive and Synergistic Two-Layer Scheduling Framework for LLM Serving (NexusSched). arXiv preprint arXiv:2509.23384 (2025).
- [6] Kwon, W., et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP* (2023).
- [7] Wu, J., et al. Unlock the Potential of Fine-grained LLM Serving via Module-wise Autoscaling (Co-CoServe). arXiv preprint arXiv:2507.18006 (2025).
- [8] Li, Z., et al. AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving. In *OSDI* (2023).
- [9] Dean, J., and Barroso, L. A. The Tail at Scale. *Communications of the ACM* 56, 2 (2013).
- [10] Ray Project. Ray Serve. (2024).
- [11] Anyscale. RayLLM: Production-Ready LLM Serving on Ray. <https://github.com/ray-project/ray-llm> (2024).
- [12] NVIDIA Corporation. Triton Inference Server. (2024).
- [13] Agrawal, A., et al. Sarathi-Serve: Efficiently Serving Large Language Models via Hybrid Scheduling. In *OSDI* (2024).
- [14] Zhong, Y., et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *OSDI* (2024).
- [15] Miao, X., et al. Towards Efficient Generative Large Language Model Serving: A Survey. arXiv preprint arXiv:2312.15234 (2023).
- [16] Lu, X., et al. Routing to the Expert: Efficient Reward-guided Ensemble of Large Language Models. arXiv preprint arXiv:2311.08692 (2023).
- [17] Wu, B., Liu, S., Zhong, Y., Sun, P., Liu, X., and Jin, X. LoongServe: Efficiently Serving Long-Context Large Language Models with Elastic Sequence Parallelism. In *SOSP* (2024). arXiv:2404.09526.
- [18] Wu, B., et al. Llumnix: Dynamic Scheduling for Large Language Model Serving. arXiv preprint arXiv:2406.03243 (2024).
- [19] Zheng, L., et al. SGLang: Efficient Execution of Structured Language Model Programs. arXiv preprint arXiv:2312.07104 (2023).
- [20] Hugging Face. Text Generation Inference. (2023).
- [21] Little, J. D. C. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* (1961).
- [22] Zheng, L., et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *NeurIPS* (2023).
- [23] NVIDIA Corporation. NVIDIA Management Library (NVML). (2024).
- [24] Kleinrock, L. *Queueing Systems, Volume 1: Theory*. (1975).
- [25] Coffman, E. G., and Denning, P. J. *Operating Systems Theory*. (1973).
- [26] Yu, G.-I., et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *OSDI* (2022).
- [27] NVIDIA Corporation. FasterTransformer. (2023).
- [28] Rasley, J., et al. DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters. In *KDD* (2020).
- [29] Wolf, T., et al. Transformers: State-of-the-Art Natural Language Processing. arXiv (2020).
- [30] Griggs, T., Liu, X., Yu, J., Kim, D., Chiang, W.-L., Cheung, A., and Stoica, I. Mélange: Cost Efficient Large Language Model Serving by Exploiting GPU Heterogeneity. arXiv preprint arXiv:2404.14527 (2024).
- [31] Anonymous. SLO-Aware Scheduling for Disaggregated Inference. arXiv preprint (2026).
- [32] Patel, P., et al. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *ISCA* (2024).
- [33] Hu, C., Huang, H., Xu, L., et al. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. arXiv preprint arXiv:2406.17565 (2024).
- [34] Qin, R., Li, Z., He, W., Zhang, M., Wu, Y., Zheng, W., and Xu, X. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. arXiv preprint arXiv:2407.00079 (2024).